National University of Computer and Emerging Sciences
Islamabad Campus

0364

Compiler Construction (CS4031)

Course Instructor(s):
Dr. Mehreen Alam, Ms. Nirmal Tariq, Ms. Aminah Ali
Section(s): A,B,C,D,E,F,G

Sessional-I Exam

Total Time (Hrs): 1
Total Marks: 45
Total Questions: 4

Date: Feb 23, 2026

Do not write below this line.

Attempt all the questions.

Q. No	1	2	3	4	Total
Marks Obt	03	11			
Max Marks	10	13	12	10	40

Instructions:

1. Show complete working where required to get full credit.
2. Attempt **Q1 and Q2** on question paper and **Q3 and Q4** on answer sheet provided. Any answer written elsewhere than provided space can not be claimed for rechecking.
3. Calculator and other resource sharing is not allowed

[CLO 3: Design and implement a simple lexical analyzer and Parser for a given CFG.]

Question 1: **[5+5=10 marks]**

a) **Consider a programming language that supports nested block comments of the following form:**

```
/* outer comment
     /* inner comment */
   end of outer */
```

In this language:

- **Comments begin with /***
- **Comments end with */**
- **Comments may be nested to arbitrary depth**

Determine whether the set of properly nested comments can be described using a regular expression. If yes write formal regex and if not give valid reason and give CFG rule if applicable.

It cannot be expressed using regex because it requires equal no. of opening and closing comment tags.

comment → /* description */

description → comment | ε

3

b) Exception handling in many programming languages (such as Java and C++) is implemented using a try–catch–finally construct. Your task is to write Context-Free Grammar for a try–catch–finally statement. In your grammar the try block should be mandatory, there must be at least one or may be multiple catch blocks, the finally block is optional. Assume that statements and ExceptionType are already defined. The general structure is:

```
try {
    statements
}
catch (ExceptionType identifier) {
    statements
}
catch (ExceptionType identifier) {
    statements
}
...
finally {
    statements
}
```

EBNF

try-catch-finally → try {statements} catch (ExeptionType identifier){ statements }
catch (ExecptionType identifier) {statements}} [finally {statements}]

[CLO 3: Design and implement a simple lexical analyzer and Parser for a given CFG.]

Question 2: **[13 marks]**

Consider the following grammar:

```
lexp     → atom | list
atom     → num |id
list     → ( lexp_seq )
lexp_seq → lexp , lexp_seq | lexp
```

a) **Left Factor this grammar where applicable: (3 marks)**

lexp → atom | list
atom → num | id
list → (lexp_seq)
lexp_seq → lexp lexp_seq'
lexp_seq → , lexp_seq | ∈

3

b) Construct First and Follow Sets for the non-terminals of resulting grammar (You can add rows as per your requirement): (10 marks)

Non-Terminals	First Sets	Follow Sets
lexp	{ num, id, (}	{ ';',), $}
atom	{num, id}	{';',), $}
list	{ (}	{';',), $}
lexp_seq	{num, id, (}	{) }

8

[CLO 3: Design and implement a simple lexical analyzer and Parser for a given CFG.]

[12 marks]

Question 3:

Consider the following grammar:

```
statement → if_stmt | other
if_stmt   → if ( exp ) stmt else_part
else_part → else stmt | ε
exp       → 0 | 1
```

and given string:

```
if (1) other else if (0) other else other
```

Find the following:

a) Syntax Tree

b) Parse Tree

c) Left Most Derivation

d) EBNF

e) Syntax Diagrams

[CLO 3: Design and implement a simple lexical analyzer and Parser for a given CFG.]

[10 marks]

Question 4:

Write down Recursive Descent Parser for following Grammar. Write a separate function for each production rule. You are not required to submit the entire program — only the functions are needed. However, any function that is called within another function must also be defined and included.

```
stmt     → if ( expr ) stmt stmtPrime | id
stmtPrime → else stmt | ε
expr     → id relop id | id
relop    → < | > | ==
id       → a | b | c
```